Session 4: Advanced Claude Code

Skills and MCPs

Brady Allardice

UPF & IBEI

Where We Are

Session 1: What Claude Code can do. How it works. CLAUDE.md and plan mode.

Session 2: The core workflow. Data cleaning, estimation, robustness, LaTeX tables.

Session 3: Git and LaTeX in VS Code. Your paper joins the project.

Session 4: The layer above the core workflow — infrastructure.

Today you will:

1. Run the usual workspace ritual — routine since session 1, quick recap below
2. Learn to route a wish — is it a **skill**, **project context**, or **MCP**?
3. Connect two MCPs (Zotero, GitHub) in practice

Today is about knowing which tool does real work and which is just novelty.

First: What Is GitHub?

GitHub is where code lives on the internet.

Every project is a *repository* — a folder with all its files plus the full history of every change.

Two copies of everything:

- **Remote** — the shared version on `github.com`
- **Local** — the version on your machine

The short version

GitHub hosts the *shared* copy. You work on your *local* copy. Git keeps them in sync.

The Vocabulary

Five words that will keep coming back today:

- `clone` — make your first local copy of a remote repo
- `pull` — fetch the latest changes *from* the remote into your local copy
- `push` — send your local changes *to* the remote
- **branch** — a parallel line of work inside the repo; changes on a branch don't affect anyone else until it's merged
- **pull request (PR)** — a proposal to merge one branch into another, with a page on GitHub for review and discussion

The flow

`clone` once. `pull` before you start. Make a **branch**, commit, `push`, open a **PR**. Someone reviews and merges.

The Course Repo

Our materials live in one GitHub repo. Everything I push goes there; everything you pull comes from there.

Let's look at the page together. Things to notice:

- The **file tree** — `session_1/`, `session_2/`, ..., `session_4/`
- The **README** — what the repo is and how to use it
- The **commit history** — every change I've made, with a message
- The **branches** — `main` is the shared one; others are work-in-progress

Why this matters for what's next

The issue we hit last session lives exactly here: you and I were both editing files in the *same* repo. To explain what went wrong, we need the picture of where things live.

The Problem We Had

Every session so far, someone has hit a merge conflict.

What we did about it:

- Deleted the local repo and recloned
- Asked Claude Code to resolve the conflict markers
- Copied work to the Desktop, wiped the directory, copied back

That worked. Everyone got unstuck. But it was a symptom fix.

The Root Cause

You were editing files *inside* the shared course repository. Every time I pushed new material, git had to reconcile your changes with mine. Conflicts were guaranteed by the structure, not by accident.

Quick Recap: Your Workspace

The layout you already know: each session is a self-contained folder.

```
claude-code-workshop/
  session_1/
  session_2/
  session_3/
  session_4/ <- today's materials live here
  ...
```

The workflow you've used since session 1: copy the session folder out of the repo into your own workspace. Work there. The shared repo stays read-only for you.

Why we do this

I can push new material anytime without breaking anything in your workspace. You can edit, break, delete, and redo without worrying about the shared state. No more conflicts.

The Three-Step Ritual

Same as every session — let's get into today's clean state:

```
# 1. Pull the latest course repo
cd ~/claude-code-workshop && git pull

# 2. Make your workspace (first time only), then copy today's folder in
mkdir -p ~/my_workspace
cp -r session_4/ ~/my_workspace/session_4/

# 3. Open your copy in VS Code and work there
cd ~/my_workspace/session_4 && code .
```

If you don't already have the course repo (you should):

`git clone https://github.com/bradyallardice/claude-code-workshop.git`

If it says it can't find the path or rejects ~

Type the commands by hand — don't paste from these slides. Pasting can turn ~ (home) and _ into look-alike characters the shell rejects. Step 2's `mkdir` creates `~/my_workspace` so the copy has

If `git pull` Fights Back

If you committed work inside the repo, `git pull` will complain. Run this recovery in the course repo, then continue with Steps 2–3:

```
git stash -u # save uncommitted edits
git branch backup-prework # save any local commits
git fetch origin
git reset --hard origin/student # match the server
git stash pop # skip if stash was empty
```

Your old commits live on `backup-prework`. If that name exists, add a suffix: `backup-prework-2`.

Claude Skills

A **skill** is a reusable instruction bundle
Claude loads on demand when a task matches.

It codifies a *convention* —

how *you* do a thing correctly —
so you don't re-explain it every session.

What a Skill Actually Does

Skills from my setup, and when each one fires:

- `latex-regression-table` — I ask for a results table. It writes a canonical CSV, renders a booktabs `.tex`, and compiles a PNG. One house style across the whole paper.
- `abstract-structure-audit` — I paste a draft abstract. It checks whether the four standard moves (puzzle, approach, findings, contribution) are present, vague, or missing.
- `paragraph-structure-audit` — I share a paragraph. It flags multiple claims per paragraph, smuggled assumptions, and weak transitions.
- `robustness-checks` — I run a regression. It proposes a structured battery (alternative SEs, subsamples, IV, event study) and executes it.

Each one is a recurring workflow done the same way every time.

How a Skill Works

A skill is a folder with a `SKILL.md` file in it.

What you set up:

```
~/.claude/skills/latex-tables/
  SKILL.md <- required
  template.tex <- optional
  examples/ <- optional
```

`SKILL.md` starts with frontmatter:

```
---
name: latex-tables
description: Use when generating a
  regression or summary-stats table.
---

# Instructions Claude follows
...
```

How Claude uses it:

1. At session start, Claude sees every skill's `name` + `description` — nothing more
2. **Two ways to invoke:** auto-call on a description match, or you call it yourself (`/latex-tables`)
3. The full `SKILL.md` body loads into context on demand
4. Claude follows those instructions

The key idea

The `description` is the auto-call trigger. Write it so Claude can tell *when* this skill applies.

Where Skills Live: The Hierarchy, Again

Remember the user → project → subfolder hierarchy from CLAUDE.md in Session 1? **Skills follow the same pattern.**

User level

```
~/.claude/skills/latex-tables/
  SKILL.md
```

Available in *every* project. For things stable *about you*.

Project level

```
your-project/.claude/skills/
  paper-style/
    SKILL.md
```

Loaded only in this project. Checked into git, shared with collaborators.

The same hierarchy keeps showing up

Hooks, **MCP servers and permissions**, and **subagents** all live at either the user or project level too. The decision is always the same: *is this stable about me, or specific to this project?*

Before You Build a Skill, Ask Where It Belongs

Three things look alike but are different tools:

1. **Skills** — reusable instruction bundles, loaded on demand
2. **Project context** — a CLAUDE.md inside your paper's directory
3. **MCPs** — connections to external systems

Most wishes of the form "I wish Claude knew X" route to one of these three.

Knowing which one saves you from building a skill that should have been something else. It also tells you when a skill is *too narrow* a tool for the job.

The Routing Rule

Skills → what's stable *about you*

Projects → what's stable *within a paper*

MCPs → what's *live*

If X changes when the project changes, it's not a skill.
If X is alive in an external system, it's not a skill either.

The Rule, Applied

What you want Claude to know	Where it belongs
Your LaTeX table conventions	Skill (stable about you)
The variables & specs of your current paper	Project context (within a paper)
Your Zotero library	MCP (live external state)
Your writing voice	Skill (stable about you)
What your current paper argues	Project context (within a paper)

Why this matters

Skills are a narrower tool than most people assume. That's a feature — narrower tools are easier to use well.

Within Skills: Two Different Jobs

Verification skills

Fire *during or after* work to check it.

Examples: LLM-mistake checker, sanity-check validator, replication auditor.

Earn keep by: catching real problems, even if used infrequently.

Production skills

Fire *during the making* of something to shape it.

Examples: LaTeX table skill, writing-style skill.

Earn keep by: frequency — every table, every document.

Why the distinction matters

"Is this worth it?" has a different answer for each. Verification skills need to catch high-cost errors. Production skills need to fire often.

Skills are for **conventions**

you hold about how to do

a thing correctly.

Conventions about table formatting. About sanity-checking data.
About responding to reviewers. About engaging with the literature.

If what you want to codify isn't a convention, it's probably not a skill.

Skills I Actually Use

Two skills I reach for regularly:

1. LaTeX tables. Encodes my house style for regression tables: significance notation, SE labels, table-notes format, journal overrides. Fires every time I generate a table. Saves ~15 min per table and keeps formatting consistent across a paper. On version three.

2. Paragraph-structure audit. One claim per paragraph, mis-cited claims, smuggled assumptions, weak transitions. Fires when I'm editing my own drafts — catches the kind of sloppiness I stop seeing after four re-reads.

What they have in common

Each codifies a *convention* I already hold. Each fires at a predictable moment — generating a table, reviewing a draft. Each has been revised through actual use.

Demo

The LaTeX tables skill, applied live.

Finding Good Use Cases Is Hard

Two skills I built but barely use:

- `auto-commit` — commits after every file edit
- `spec-validator` — pre-estimation data and spec checks

What they have in common: each sounded useful in the abstract. Each was built on an idea of how research *should* go. Each got replaced by a one-off prompt the first time I actually needed the job done.

The specificity can also work against you — a tightly scoped skill only fires when the exact shape of the task matches, and it's easy to build one whose niche never actually shows up.

Strong recommendation

Do not go build five skills this weekend. You will not use most of them.

Instead: **build one.** Try it on your current work. Tweak it where it fires wrong. Let it earn its place in your workflow. *Then* add another.

How to Build One That Works

What separates a skill that fires right from one that sits unused:

- **Write the description for the trigger, not the reader.** Claude decides whether to auto-call on the description alone. Name the task, the keywords, and what the skill does *not* handle.
- **Keep it narrow.** One genre of table, one kind of audit, one workflow. Broad skills fire at the wrong moment, or never.
- **Encode your convention, not generic knowledge.** The `SKILL.md` should capture the judgment calls *you* make. Claude already knows boilerplate.
- **Ship a reference example.** An `examples/` folder with a real working artifact gives Claude something concrete to pattern-match on.
- **Revise after the first real use.** Version one fires wrong somewhere. Fix the trigger or the instructions, retry.

Let's Build One: `tex-reference-audit`

The need: before submission, I want to know every `\cite`, `\ref`, and `\label` in the manuscript actually resolves.

What it checks:

- Every `\cite{key}` resolves to a `.bib` entry
- Every `.bib` entry has required fields (author, year, title, journal, pages...)
- Every `\ref{label}` points to an existing `\label`
- Every `\label` is actually referenced (no orphans)
- Figures use `fig:`, tables use `tab:`

The description — copy, paste, build live:

```
---
name: tex-reference-audit
description: Use when reviewing
  or finalizing a LaTeX manuscript.
  Audits \cite against the .bib
  file, \ref/\label pairs for
  figures and tables, flags
  missing fields, duplicates, and
  orphans. Trigger: "check refs,"
  "audit citations," "review
  LaTeX."
---
```

No script — the body is just instructions Claude follows to parse, check, and report.

Exercise: Build a Skill (25 min)

Working solo. Each of you walks out with one skill installed and tested.

1. **Pick a convention (3 min).** Something you actually did in the last two weeks — a format you enforce, an audit you redo by hand, context you retype. Not a hypothetical.
2. **Route it (2 min).** Confirm with your neighbor: is this *really* a skill? Not project context, not an MCP. If a skill, is it **verification** or **production**?
3. **Write `SKILL.md` (10 min).** A clear `name`, a `description` that tells Claude *when* to fire, and the instructions for when it does.
4. **Install (1 min).** Save to `~/.claude/skills/<your-skill>/SKILL.md`.
5. **Try it (9 min).** Open a *fresh* Claude Code session. Ask for a task that should trigger it. If it doesn't fire, the `description` is wrong — fix and retry.

Deliverable

A working skill, installed and fired at least once.

What you leave with:

- A folder at `~/.claude/skills/<your-skill>/` with a real `SKILL.md`
- One transcript where Claude triggered the skill on a real task
- One concrete thing you'd change now that you've seen it run

Two or three people share: *What is it? Did it trigger on the first try? What surprised you?*

The goal

You leave having built one thing, seen it fire, and learned where the `description` is too vague or too narrow. That is the empirical loop. Everything else is speculation about skills you haven't tried.

Skills: The Arc

You now have everything you need to make skills work for you:

1. A **routing rule** to spot a real candidate — stable-about-you, not within-a-paper, not live
2. A **typology** — verification vs. production — to predict when and how often it'll fire
3. A **heuristic** — skills codify the conventions you already hold
4. An **empirical loop** — build, fire, refine, which you just ran end-to-end
5. **One skill already installed** on your machine

The durable takeaway

Every convention you hold about how to do a thing correctly is a candidate skill waiting to be written down. The ones that earn their place will compound across every project you work on — and you now know how to tell the good candidates from the noise.

Connecting Claude Code
to systems outside your project.

From the routing rule: *MCPs are for what's live.*
Here are two live things worth connecting.

Forty-five minutes. Two MCPs. One pair exercise.

What an MCP Is

MCP = Model Context Protocol. An open standard for plugging external tools into Claude Code.

- Each MCP is a separate server.
- Claude Code speaks to it over a standard protocol.
- The server exposes *tools* (like `zotero_search`) that Claude Code can call.

Without MCPs: Claude Code reads and writes files in your project. That's it.

With MCPs: it can query your Zotero library, open a GitHub PR, read a database, post to Slack — anything someone has built a server for.

The framing

MCPs turn Claude Code from a tool that operates on your *files* into a tool that operates on your *workflow*.

Where Skills and MCPs Compose

The paragraph-structure skill is the first one you've seen that invokes an MCP.

Alone:

- *Skill only:* "This claim needs a citation. Figure one out."
- *MCP only:* "Here's your Zotero library." But nothing knows when to search.

Composed:

- The skill knows *when* a claim is mis-cited or uncited
- The MCP returns actual papers from your library that would support it
- The MCP constrains the skill to real papers — no hallucinated citations

The pattern

A skill knows *when* to act; an MCP provides *live data*. Together, they do what neither can alone.

The Two-Reason Rule

Once you've routed something to an MCP, ask: which of two reasons?

1. **Extend capability.** Let Claude Code do something it couldn't do at all.
   Example: open a GitHub pull request. Query your Zotero library. Run a SQL query against your database.

2. **Reduce friction at a boundary.** Let it do something you *could* already do, but where context-switching between tools kills the flow.
   Example: inserting citations while drafting. You could look them up in Zotero and paste. But stopping to do that breaks writing flow.

Today I'm showing two specific MCPs I think are worth adopting. There are dozens more. The *framework* is what transfers; the two cases are how you learn it.

Callback to the meta-lesson: MCPs are easy to accumulate. Ask what each one earns.

The Friction Case: Zotero

Drafting with citations normally looks like:

1. Write a sentence that needs a citation
2. Stop. Open Zotero. Search. Find the paper. Copy the cite key.
3. Come back to the editor. Paste. Context-switch back into drafting.
4. Repeat ten times per paragraph.

With the Zotero MCP: "Find me the Smith 2019 paper on voter ID, add it to refs.bib, and cite it here." Claude Code does the search, the BibTeX export, the `\cite{}` insertion — without you leaving the `.tex` file.

Why it earns its keep

Reduce friction at a boundary. You could already do all of this manually. The MCP just removes the tool-switch.

What the Zotero MCP Can Do

The Zotero MCP exposes around two dozen tools. They fall into two buckets:

Read from your library

- **Search:** keyword, semantic (natural language), by tag, advanced filters
- **Browse:** collections, recent items, saved searches, RSS feeds
- **Pull metadata:** title, authors, abstract, BibTeX fields
- **Pull full text** of attached PDFs
- **Read** your existing notes and annotations

Write back to your library

- **Add papers** by DOI, URL, or local PDF (auto-fetches metadata + open-access PDFs)
- **Create notes** attached to items
- **Create and organize collections**
- **Update item metadata** — tags, titles, abstracts

What this unlocks

Claude can read the *actual full text* of papers in your library — not just the title — so it can reason about what's *in* the paper, not just whether the citation exists.

Zotero MCP: Live Demo

A single prompt:

```
Find all papers in my claude_code_zotero collection
on local economic voting, and for each give me the
abstract and any notes I have on it.
```

What you'll see in the tool-call log:

1. `zotero_get_collection_items("claude_code_zotero")` — lists the collection
2. `zotero_semantic_search("local economic voting")` — filters to the topic
3. `zotero_get_item_metadata(...)` — pulls title, authors, abstract
4. `zotero_get_notes(...)` — pulls my existing notes on each item

Watch the tool calls, not just the output. Seeing which tools fire tells you what the MCP actually does.

Exercise: Mini Literature Review (15 min)

You installed the Zotero MCP as Session 4 prework and pointed it at the shared group library `claude_code_zotero` **— nine papers on economic voting.**

Your task. Draft a **five-sentence literature review** on *one* of:

- How economic conditions shape vote choice
- Whether voters punish incumbents for bad economies
- Whether voters respond to the *local* economy versus the national one

Rules:

- Every sentence cites at least one paper from `claude_code_zotero`
- Only use cite keys that *actually exist* in the collection — no hallucinations
- Claude does the searching, full-text reading, and drafting

What to Watch For

While the model works, watch the tool calls:

- Did it run `zotero_semantic_search`, or skip search and guess?
- Did it actually pull `zotero_get_item_fulltext`, or cite from titles alone?
- Do the claims in your sentences match what the papers actually say?

After drafting, spot-check one citation. Open the paper in Zotero. Does the sentence you wrote accurately represent what the paper argues?

The real lesson

An MCP gives Claude *access*. It does not give Claude *judgment*. The failure mode to expect: a plausible sentence citing a paper that says something adjacent — but not quite that. **You still read.**

How GitHub Works as a Collaborator

Session 3 was the solo loop:

`commit` → `instruct Claude` → `diff` → `keep or revert`

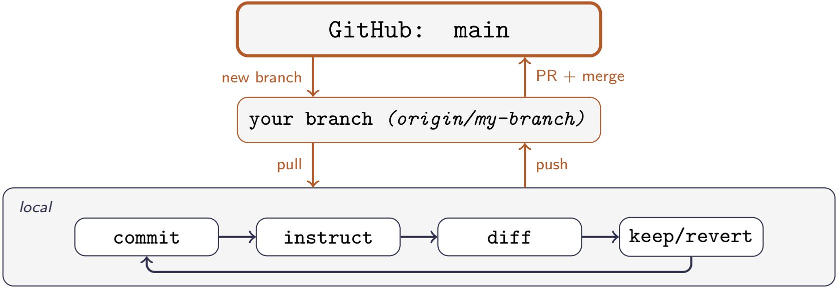
One person, one copy. Everything happens on your machine.

Add a coauthor and two new problems appear:

1. **Parallel work.** You both want to edit at once. Whose version wins?
2. **Combining work.** Your changes live on your laptop, theirs on theirs. How do they become one shared version?

GitHub's answer uses two moves: branches and pull requests.

GitHub's Answer: Branches + PRs



- `main` is the shared source of truth. Changes only return via PR + merge.
- **Your branch** is a parallel lane on GitHub — your work stays separate until reviewed.
- **Locally** is where the solo loop runs, on the files of your checked-out branch.

Working With Branches: The Commands

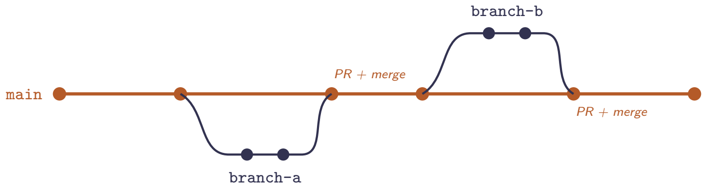
The whole branch workflow in five moves:

```
# 1. Start from an up-to-date main
git checkout main
git pull
# 2. Create a new branch
git branch my-branch-name
# 3. Switch to it
git checkout my-branch-name
# 4. Commit
git add <files>
git commit -m "..."
# 5. Push (first time uses -u; later pushes: git push)
git push -u origin my-branch-name
```

`git branch` lists branches (∗ marks current). Commit before switching — uncommitted edits follow you between branches.

What Is a Pull Request?

A pull request (PR) is the proposal to merge one branch into another. Everyone branches off `main`, works independently, then opens a PR to rejoin.



On the PR page: a diff, a conversation thread, and an approve-and-merge button gated by the drafter's assigned **reviewer**.

On a collaborative repo, every PR has an assigned reviewer. They have to click approve *and* merge — nothing reaches `main` until they do.

Collaboration, Live

Time to see the whole loop on a real repo. I'll walk through branches, commits, push, PR, review, and merge — on `github.com/bradyallardice/github_collaboration_practice`.

What I'll do: write a list of characters for a story. You pick which story:

1. **The Conference From Hell** — every session, break, and hallway moment at an academic conference goes sideways
2. **The Dissertation That Wrote Itself** — a PhD student's dissertation has gained sentience; each chapter is one night of it taking new liberties
3. **The Department Mouse** — a very literate mouse is stealing specific things from the department; it appears to have a plan

Watch the steps. Ask questions. I'll end with the cast pushed to `main`.

Before We Go Further: GitHub Access

Two quick things:

1. **If you haven't already emailed me your GitHub username, write it on the paper going around now.**
2. **Everyone: check your email and accept the collaboration request for the class repo.** Without it you can't push a branch or open a PR.

The invite appears as an email from GitHub, and also as a banner on the repo page if you're logged in. Click **Accept invitation**.

One minute. Shout if it doesn't show up — I'll re-send.

. . . Now You're Going to Write the Story

That repo I just pushed to? You're going to write the rest of it.

Each of you claims one of the 16 chapter slots, featuring the cast I just pushed. Write, branch, PR, merge your own chapter — all the way through. At the end, we assemble every chapter and read the story out loud.

Your chapter should feature at least one of the named characters. Beyond that, go wherever it takes you.

The writing is short on purpose. The point is the Git workflow, not the prose.

Your Chapter

I've assigned each of you a specific chapter. Open `CHAPTERS.md` at the root of the class repo — your name is next to a numbered slot. That's your chapter.

Then you run the whole loop yourself:

- Branch from `main`, write your chapter at `chapters/chapter_N.md`
- Commit, push, open a PR
- Approve and merge your own PR

Ground rules

- **Length doesn't matter.** A paragraph is fine.
- **Don't try to match other chapters.** You can't — everyone is writing simultaneously.
- **Don't polish.** Five minutes. Write, push, merge, move on.

1. Clone the repo, open it, branch, write, commit, push:

```
git clone https://github.com/bradyallardice/github_collaboration_practice.git
cd github_collaboration_practice
code . # open the repo in VS Code
git checkout -b chapter-N
# write your assigned chapter at chapters/chapter_N.md
git add chapters/chapter_N.md
git commit -m "Chapter N"
git push -u origin chapter-N
```

2. Open and merge the PR on GitHub:

1. Click the **Compare & pull request** banner, base = `main`
2. **Create pull request**
3. **Merge pull request** → **Confirm merge**

Something break? → Troubleshooting GitHub

What to Expect

Some of you will finish. Some will not. Both are fine.

The goal is that *everyone sees the loop end-to-end*: branch, push, PR, merge. Even if you need help at every step, you will recognize the shape next time.

What will go wrong (for some of you):

- **Push rejected** because `main` moved. Run `git pull --rebase origin main` and push again.
- **PR opened against the wrong base.** Edit the base branch in the GitHub UI, or close and reopen.
- **You get stuck for more than 90 seconds.** Flag me. Don't silently spin.

After all PRs are merged, I'll run the assembly script and read us the story.

What the GitHub MCP Adds on Top

You just did the whole collaboration loop with plain git. The GitHub MCP doesn't replace any of that — it lets Claude Code *reach* the parts that live on GitHub, not just on your disk:

- Open a pull request from a branch you've been working on
- Leave review comments on a coauthor's PR
- **Approve and merge a PR** — including its own
- Create issues to track tasks you don't want to lose
- Push files to a remote repository

Why it earns its keep

Extend capability. "Open a PR" and "merge it" aren't things Claude Code can do without this. It's a genuine new capability, not a friction fix.

Demo: Finish the Story with the MCP

All your chapters are on `main`. Watch me close the story out — Claude Code does the whole thing through the MCP.

A single prompt:

```
Pull the latest main, read every chapter in chapters/,
write a short conclusion that ties the threads together,
commit it on a branch, open a PR, and merge it.
```

What you'll see in the tool-call log:

1. `git pull` — bring main up to date
2. Read each chapter file, then `Write` the conclusion
3. `git commit` + `git push` on a new branch
4. `mcp__github__create_pull_request`
5. `mcp__github__merge_pull_request` — the autonomous one

MCP as Helper, Not Substitute

Did you notice what just happened? One prompt. One merge into `main`. No second pair of eyes.

The risk the MCP unlocks

The GitHub MCP can approve and merge PRs autonomously. If you say "handle the PR," Claude can push to `main` with no human review — fine for a playful class story, dangerous on a real research repo.

How to stay safe:

- **Turn on branch protection** — a GitHub setting (Repo → Settings → Branches) that refuses to merge a PR unless it has at least one approval from *someone other than the author*. The MCP has to obey it, same as the web UI.
- **Scope what you let Claude do.** "Open a PR" is fine; "merge it" should make you pause.
- **Watch the tool-call log** when the MCP acts on branches.

The exercise *asked for* two humans — that was a convention, not a GitHub rule. By default, anyone with write access (including Claude via the MCP) can merge a PR alone. The branch-protection setting is what actually makes human review mandatory.

MCPs: The Meta-Lesson, Again

GitHub MCP is optional infrastructure. The underlying git workflow is not.

If you find yourself reaching for the MCP to fix a conflict — sometimes that's the right move. But sometimes (like this morning's session-folder fix) it's papering over a structural problem.

Before adopting an MCP, ask

1. Does this genuinely extend what I can do, or reduce real friction?
2. Will I invoke it more than three times a month?
3. Am I using it because it helps, or because it's new?

Most MCPs fail question 2. The two today pass all three — *for me*. Your answers may differ.

Session 5 covers three things:

1. **Subagents** — a focused Claude spawned for a bounded job: parallelism plus context hygiene, then scaled up into named, reusable specialists.

2. **Hooks** — shell commands that run automatically on events (file writes, session start, tool use), for enforcement that doesn't depend on Claude remembering.

3. **The final project** — two-person teams create an empty GitHub repo, copy in `session_5/capstone-template/`, read the paper, use Zotero, then write a short replication + extension note.

What You Built Today

1. **A cleaner workflow.** You're working in your own workspace. No more merge conflicts against the shared repo.
2. **A framework for skills.** A routing rule (stable-about-you / within-a-paper / live), a verification vs. production typology, and one skill already installed on your machine.
3. **Two MCPs on your map.** Zotero (friction) and GitHub (capability). The two-reason rule to evaluate others.

The thread running through

Agents solve symptoms well. Your job is to notice when the structure is wrong, when the skill is ceremony, and when the MCP is novelty.

Next Session + Prework

Session 5: Subagents, Hooks, and the Final Project.

- **Subagents** — when to spawn one, then how to define named, reusable specialists with their own scope and tools
- **Hooks** — automatic shell commands on events (file writes, tool calls, session start)
- **Final project** — two-person teams create an empty GitHub repo, copy in `session_5/capstone-template/`, then do paper familiarization, Zotero-backed literature review, replication, extension, and reflection

Before next session:

1. **Check GitHub:** make sure you can create a repo, add a collaborator, and push a branch
2. **Check Zotero:** Session 5 imports `zotero/capstone_library.bib` with PDFs from `zotero/pdfs/`
3. **Run the routing test** on one thing you wished Claude already knew — is it a skill, project context, MCP, hook, subagent, or just a one-off prompt?
4. **Try one thing from today** on your actual research: a skill, an MCP, or just the new repo workflow

Bring a question, a frustration, or a small win from the week. Session 5 opens with those.

Troubleshooting: When GitHub Fights Back

Common failure modes during the exercise — and the fix.

- **Authentication fails on first push.** Run `gh auth login` and pick HTTPS + browser. (`brew install gh` first if needed.)
- **"Updates were rejected"** — `main` moved while you worked.
  `git pull --rebase origin main && git push`
- **Conflict markers (<<<<<<<) in a file.** Open it, edit to the version you want, delete the markers. Then `git add <file>` and `git rebase --continue` (or `git commit` if merging). Walkthrough →
- **You committed to `main` by accident.** Save the work to a new branch, then rewind `main`:

```
git checkout -b chapter-N
git checkout main && git reset --hard origin/main
git checkout chapter-N
```

- **PR opened against the wrong base.** Edit the base branch in the GitHub UI at the top of the PR — no need to close and reopen.
- **Stuck for more than 90 seconds.** Flag me. Don't silently spin — the fix is usually one command.

Anatomy of a Merge Conflict

When git can't auto-combine two versions of the same lines, it leaves the file marked up like this:

```
The keynote speaker took the stage.
<<<<<<< HEAD
He launched into a 90-minute monologue about LLMs.
=======
She fumbled with the slide remote for two full minutes.
>>>>>>> origin/main
The audience applauded politely.
```

Three parts:

- `<<<<<<< HEAD` — start of **your** version (the branch you're on)
- `=======` — divider
- `>>>>>>> origin/main` — end, plus the name of the **incoming** version

Git isn't broken. It just can't tell which version you want — and is asking you to decide.

Resolving a Conflict by Hand

1. **Find them:** `git status` *(conflicted files appear under "both modified")*
2. **Open each one. Edit until only the version you want remains — no markers.**

Before:

```
<<<<<<< HEAD
He launched into a 90-min
monologue about LLMs.
=======
She fumbled with the slide
remote for two full minutes.
>>>>>>> origin/main
```

After (keep one, the other, or merge):

```
She fumbled with the slide
remote for two full minutes,
then launched into a 90-min
monologue about LLMs.
```

3. **Stage the file and continue.**

```
git add chapters/chapter_3.md && git rebase --continue
# (or: git commit, if you were merging instead of rebasing)
```

Want to bail out? `git rebase --abort` (or `git merge --abort`) reverts to the pre-conflict state.

Letting Claude Resolve It

Claude Code is good at the mechanical part — finding the markers and proposing a sensible merge.

A prompt that works:

```
There's a merge conflict in chapters/chapter_3.md.
Read both sides, propose a resolution that keeps the
intent of both, and apply it. Show me the diff first
- do not commit.
```

Safe to delegate:

- Trivial conflicts (formatting, adjacent edits)
- One side is clearly the newer or correct version
- Prose where the diff is short and you can read it end-to-end

Do it yourself:

- Code where behavior changes — you need to understand both sides
- Numerical results, tables, anything you'd want to verify line-by-line
- Conflicts touching more than a handful of files

Always read the diff. "Resolved" is not the same as "correct."